

CO4 – ASSESSMENT TASK 2

INDUSTRY PROBLEM SOLVING TASK

Constraint-Based Selection of Computer Vision Detection Methods

Autonomous Vehicle Perception • Smart Factory Defect Inspection

Prepared by: Nitheesh Kondapalli

B.Tech – Artificial Intelligence & Machine Learning

Saveetha School of Engineering (SIMATS)

Course Outcome: CO4 – Data Interpretation & Applied Decision-Making in Computer Vision

Contents

1. Introduction and Objective
2. Background: Overview of the Detection Methods
3. Problem Statements (as given)
 - 3.1 Autonomous Vehicle Vision System Selection
 - 3.2 Smart Factory Product Inspection Selection
4. Answer 1 – Autonomous Vehicle Vision System
5. Answer 2 – Smart Factory Product Inspection
6. Comparative Summary Across Both Case Studies
7. Recommendations and Best Practices
8. Conclusion
9. References

1. Introduction and Objective

Selecting the right computer-vision detection method for a real-world industrial system is rarely about picking the single most accurate algorithm available. Real deployments operate under hard operational limits — a maximum acceptable delay, a minimum acceptable accuracy, and a fixed hardware budget — and a method that violates even one of these limits is not deployable, however impressive its other metrics look on paper. This report works through two such industry scenarios: an autonomous vehicle's onboard object-detection system and a smart factory's automated defect-inspection system. In both cases, three candidate detection methods are evaluated against clearly stated constraints, and a single method is logically justified as the correct choice.

The objective of this task is twofold: first, to demonstrate a structured, constraint-driven decision-making process (restate constraints → check every method against every constraint → eliminate violators → justify the survivor); and second, to build intuition for why certain families of algorithms (classical, real-time deep learning, and heavyweight deep learning) tend to occupy predictable positions on the accuracy-speed-cost spectrum, and why that pattern repeats across very different industries.

2. Background: Overview of the Detection Methods

Before evaluating the two case studies, it is useful to understand what each family of methods actually is, because their constraint behaviour follows directly from their underlying design.

2.1 Viola-Jones / Traditional Image Analysis

The Viola-Jones framework (and traditional image-analysis pipelines built on similar principles) detects objects using hand-crafted features — such as Haar-like features, edge gradients, or simple intensity thresholds — combined with a cascade of lightweight classifiers. Because these features are simple and fixed rather than learned, the computation per frame is very low, which is why such methods are extremely fast and can run on minimal hardware, even without a GPU. The trade-off is that hand-crafted features generalize poorly to variation: changes in lighting, angle, occlusion, texture, or object appearance that were not explicitly anticipated by the designer tend to reduce accuracy sharply. This is why, in both case studies below, this family of method is the fastest and cheapest but consistently falls short of the accuracy threshold.

2.2 YOLO (You Only Look Once)

YOLO is a real-time deep-learning object detector that treats detection as a single regression problem, predicting bounding boxes and class probabilities directly from the full image in one forward pass through a convolutional neural network. This single-pass design is what gives YOLO its defining characteristic: near deep-learning-level accuracy at a fraction of the latency and computational cost of larger, multi-stage deep networks. It was explicitly designed for real-time applications (traffic monitoring, robotics, live video analytics), which is precisely the category both case studies fall into. Its moderate computational requirement makes it deployable on embedded GPUs or moderate edge-computing hardware — exactly the profile described in both problems.

2.3 Deep Learning (Large CNN / Transformer-based Detectors)

This category refers to larger, deeper, and often multi-stage detection architectures (e.g., deeper CNN backbones, region-proposal networks, or transformer-based detectors) that prioritize maximum accuracy over speed. Their depth and richer feature hierarchies allow them to capture subtle patterns, tolerate occlusion, and generalize across difficult conditions better than any lighter method — which is why they post the highest accuracy figures in both scenarios below. However, that same depth means significantly more computation per frame, longer inference time, and a requirement for high-end hardware (often dedicated GPUs/accelerators). In latency- or cost-constrained real-time systems, this makes them a poor fit despite being the most “accurate” option in isolation.

2.4 General Positioning of the Three Families

Across both problems, a consistent pattern emerges: the traditional/classical method is fast and cheap but accuracy-limited; the heavyweight deep-learning method is the most accurate but slow and resource-hungry; and YOLO-class real-time deep learning sits in between, typically close enough to the heavyweight method's accuracy while remaining fast and light enough for real-time, resource-constrained deployment. This is exactly why constraint-driven industrial decisions so often converge on YOLO-class detectors, as the analysis in Sections 4 and 5 will show quantitatively.

3. Problem Statements (as given)

3.1 Autonomous Vehicle Vision System Selection

An automobile company is developing an autonomous vehicle detection system that must operate under the following conditions:

- If accuracy < 90% → system fails
- If latency > 70 ms → unacceptable
- The vehicle supports moderate computational resources

Methods available:

- Viola-Jones: 82% accuracy, 30 ms latency, low computation
- YOLO: 93% accuracy, 60 ms latency, moderate computation
- Deep Learning: 97% accuracy, 90 ms latency, high computation

Task

Compare the three methods and evaluate which method satisfies all the constraints. Justify your decision using logical reasoning based on accuracy, latency, and computational requirements.

3.2 Smart Factory Product Inspection Selection

A manufacturing company wants to deploy a real-time computer vision system to detect defective products on a production line. The system must operate under the following conditions:

- If accuracy < 95% → defects may be missed
- If processing time > 50 ms → production is affected
- Hardware supports moderate computation

Methods available:

- Traditional Image Analysis: 91% accuracy, 25 ms, low computation
- YOLO: 96% accuracy, 45 ms, moderate computation
- Deep Learning: 98% accuracy, 75 ms, high computation

Task

Compare the three methods and evaluate which method satisfies all the constraints. Justify your decision using logical reasoning based on accuracy, processing time, and computational requirements.

4. Answer 1: Autonomous Vehicle Vision System Selection

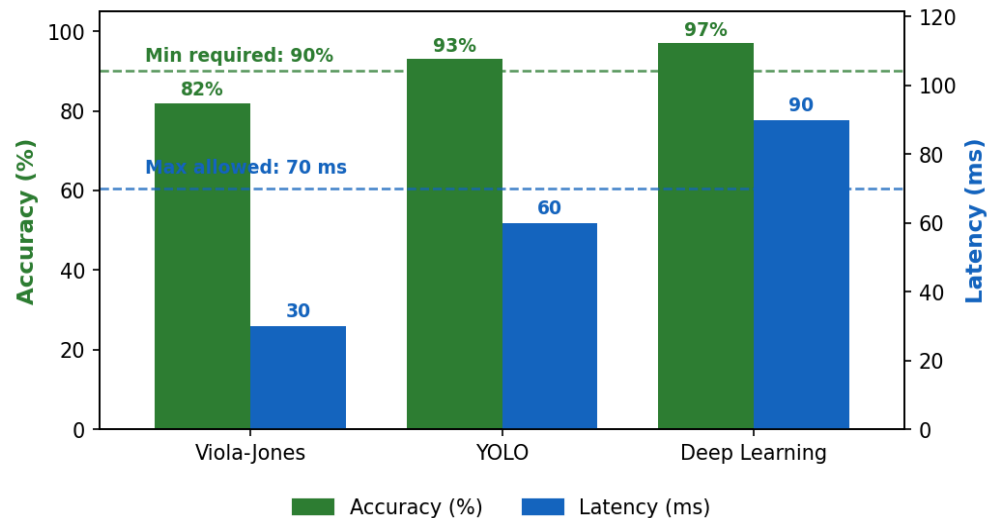
4.1 Restate the constraints

- Constraint 1: Accuracy must be $\geq 90\%$ (below this, the system fails)
- Constraint 2: Latency must be ≤ 70 ms (above this, it is unacceptable)
- Constraint 3: The vehicle only supports moderate computational resources

4.2 Check every method against every constraint

Method	Accuracy ($\geq 90\%$)	Latency (≤ 70 ms)	Computation Fit	Overall
Viola-Jones	82% → Fail	30 ms → Pass	Low (fits)	Rejected
YOLO	93% → Pass	60 ms → Pass	Moderate (exact fit)	Selected
Deep Learning	97% → Pass	90 ms → Fail	High (exceeds vehicle capacity)	Rejected

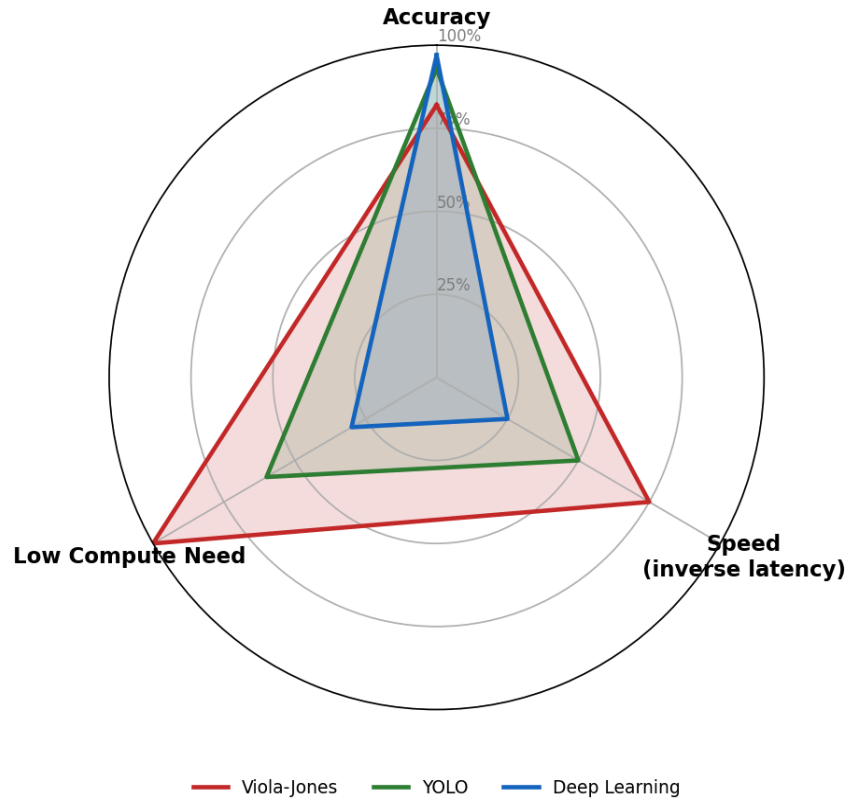
Q1: Autonomous Vehicle Vision System — Accuracy vs Latency



4.3 Multi-criteria visual comparison

The radar chart below normalizes all three criteria — accuracy, speed (inverse of latency), and low compute need — onto a common 0–100% scale so the trade-off shape of each method can be seen at a glance. A method that is well-balanced across all three constraints (rather than dominant in only one) is generally the safer real-world choice, and the chart makes clear that YOLO is the most balanced of the three, without the extreme weakness that disqualifies the other two.

Q1: Autonomous Vehicle — Multi-Criteria View



4.4 Reasoning for each method

Viola-Jones has the lowest latency (30 ms) and the lightest computational footprint, which look attractive at first glance. However, it fails the very first and most critical constraint: its accuracy (82%) is below the 90% threshold, meaning the system would fail outright. In an autonomous vehicle, undetected or misclassified obstacles directly threaten safety, so an accuracy failure cannot be compensated for by good speed — it is automatically disqualified regardless of its other strengths.

Deep Learning delivers the highest accuracy (97%), comfortably clearing the 90% requirement, and would be the most desirable option from a pure detection-quality standpoint. However, its latency (90 ms) exceeds the 70 ms limit, and it demands high computational resources beyond what the vehicle supports. A 90 ms delay in a fast-moving vehicle translates into a real physical distance travelled “blind” before the system reacts, which is unacceptable for collision avoidance. It is therefore disqualified despite its superior accuracy.

YOLO is the only method that clears every constraint simultaneously: 93% accuracy exceeds the 90% minimum, 60 ms latency is within the 70 ms limit, and it requires only moderate computation — exactly matching the vehicle's available resources.

4.5 Sensitivity check — how much safety margin does YOLO have?

It is also worth checking how close YOLO sits to each boundary, since a method that only barely satisfies a constraint offers little safety margin for real-world variability (e.g., a busy scene with many overlapping objects, or a temporary hardware slowdown). YOLO's accuracy (93%) sits 3 percentage points above the 90% minimum — a modest but workable margin — while its latency (60 ms) sits 10 ms below the 70 ms ceiling, giving roughly a 14% latency buffer before the constraint would be violated. This buffer is important in practice, since real-world driving conditions (rain, glare, dense traffic) typically push latency and accuracy slightly worse than clean benchmark numbers, so a method with zero margin would be risky even if it “passes” on paper.

4.6 Decision and Justification

YOLO should be selected for the autonomous vehicle vision system. It is the only method that simultaneously satisfies all three hard constraints (accuracy, latency, and computational capacity), which is a strict, non-negotiable requirement in this problem — a method that fails even one constraint is not viable, no matter how well it performs on the others. YOLO's design, which processes detection in a single fast network pass, makes it well suited to real-time, safety-critical environments like autonomous driving, giving the vehicle enough reaction time while still detecting objects reliably enough to keep the system operating safely within its resource budget.

5. Answer 2: Smart Factory Product Inspection Selection

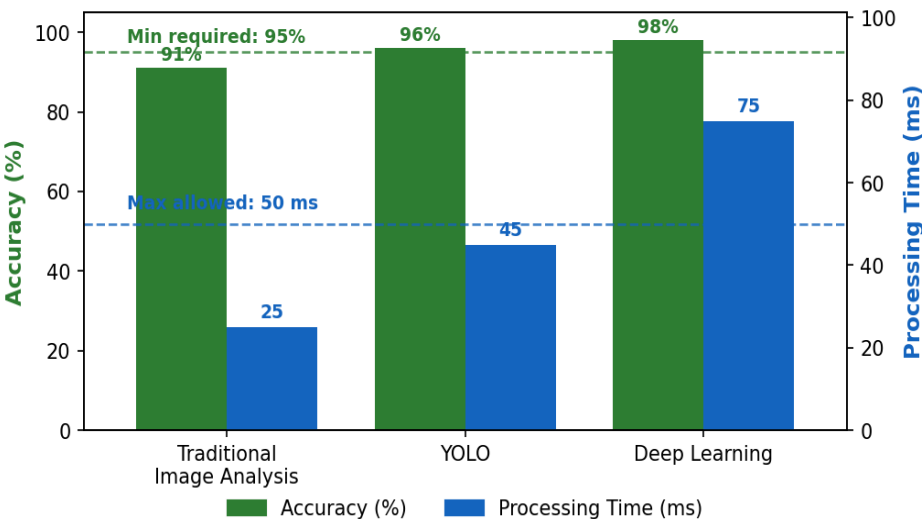
5.1 Restate the constraints

- Constraint 1: Accuracy must be $\geq 95\%$ (below this, defects may be missed)
- Constraint 2: Processing time must be ≤ 50 ms (above this, production is affected)
- Constraint 3: Hardware only supports moderate computation

5.2 Check every method against every constraint

Method	Accuracy ($\geq 95\%$)	Time (≤ 50 ms)	Computation Fit	Overall
Traditional Image Analysis	91% → Fail	25 ms → Pass	Low (fits)	Rejected
YOLO	96% → Pass	45 ms → Pass	Moderate (exact fit)	Selected
Deep Learning	98% → Pass	75 ms → Fail	High (exceeds hardware)	Rejected

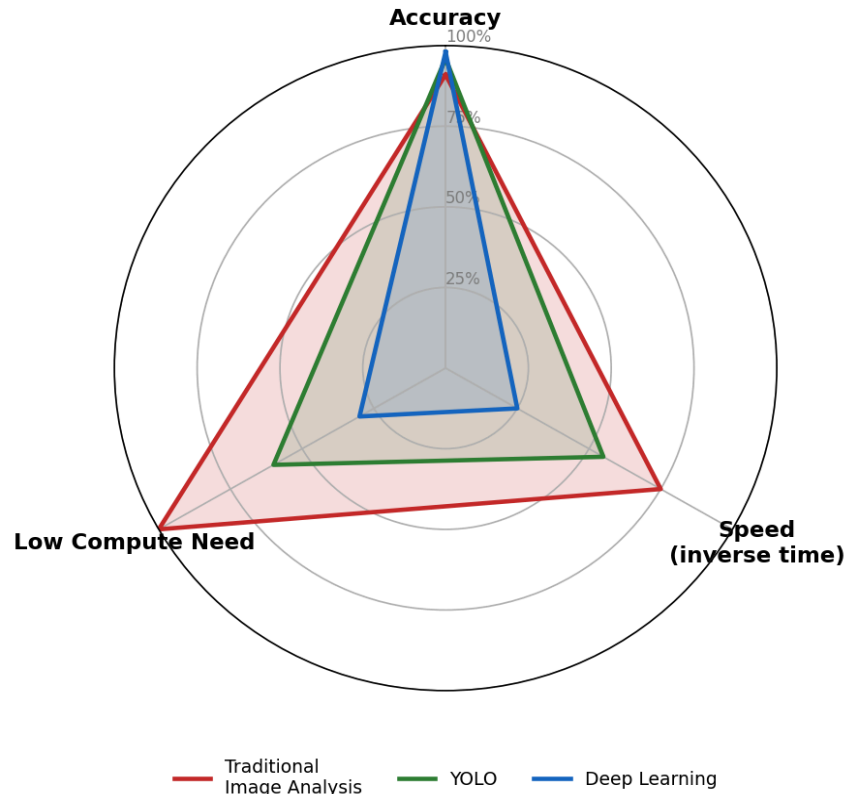
Q2: Smart Factory Product Inspection — Accuracy vs Processing Time



5.3 Multi-criteria visual comparison

As with the first case study, normalizing accuracy, speed, and computational lightness onto a single radar chart shows the same underlying pattern: Traditional Image Analysis is strong only on speed and cost, Deep Learning is strong only on accuracy, and YOLO is the only method with a reasonably balanced profile across all three axes — which is exactly what a strict, simultaneous, three-constraint requirement demands.

Q2: Smart Factory — Multi-Criteria View



5.4 Reasoning for each method

Traditional Image Analysis is fast (25 ms) and computationally cheap, but its accuracy (91%) falls short of the 95% requirement. On a production line, this means defective products are more likely to slip through undetected, which directly undermines the purpose of the inspection system — a quality-control system that misses defects defeats its own goal, so it is disqualified regardless of its speed advantage.

Deep Learning achieves the highest accuracy (98%), well above the 95% requirement, which would minimize missed defects better than any other option. However, its processing time (75 ms) exceeds the 50 ms limit, and it requires high computation that the factory hardware does not support. On a real-time production line, exceeding the allowed processing time causes a bottleneck — products would queue up faster than the system can inspect them, slowing or halting production. It is therefore disqualified despite its top-tier accuracy.

YOLO satisfies every constraint at once: 96% accuracy exceeds the 95% minimum, 45 ms processing time is within the 50 ms limit, and its moderate computational requirement matches the available hardware exactly.

5.5 Sensitivity check — how much safety margin does YOLO have?

YOLO's accuracy (96%) sits 1 percentage point above the 95% minimum — a comparatively tight margin that would warrant ongoing monitoring in production (e.g., periodic re-validation as product designs or lighting on the line change), while its processing time (45 ms) sits 5 ms under the 50 ms ceiling, a 10% buffer. This is a narrower safety margin than in the autonomous-vehicle case, which is a useful reminder that “passes the constraint” and “passes the constraint comfortably” are different levels of confidence, and the factory may want to periodically audit YOLO's real-world accuracy to ensure it does not drift below 95% as conditions on the line evolve.

5.6 Decision and Justification

YOLO should be selected for the smart factory product inspection system. Like in the autonomous vehicle case, all three constraints are strict pass/fail thresholds, so any method that violates even one — however strong it is elsewhere — is not deployable. Traditional Image Analysis fails on accuracy (too many defects would be missed) and Deep Learning fails on speed and hardware fit (it would slow production and exceed the available compute). YOLO alone clears every threshold, giving the factory a system that is accurate enough to reliably catch defects, fast enough to keep up with the production line, and light enough to run on the existing moderate-capability hardware — making it the only practical, constraint-satisfying, and cost-effective choice.

6. Comparative Summary Across Both Case Studies

Although the two case studies come from very different industries — automotive safety and manufacturing quality control — the underlying decision structure and outcome are strikingly similar. The table below places both problems side by side to highlight the pattern.

Aspect	Autonomous Vehicle (Q1)	Smart Factory (Q2)
Accuracy threshold	$\geq 90\%$	$\geq 95\%$
Latency / time threshold	≤ 70 ms	≤ 50 ms
Hardware profile	Moderate	Moderate
Classical method result	Fails on accuracy (82%)	Fails on accuracy (91%)
Deep Learning result	Fails on latency (90 ms) & hardware	Fails on time (75 ms) & hardware
YOLO result	Passes all constraints	Passes all constraints
YOLO's accuracy margin	+3 points above minimum	+1 point above minimum
YOLO's speed margin	~14% under latency ceiling	~10% under time ceiling
Final decision	YOLO	YOLO

The recurring pattern is clear: the classical/traditional method is consistently disqualified by accuracy, the heavyweight deep-learning method is consistently disqualified by latency and hardware fit, and YOLO is the only method architecturally positioned to satisfy tight real-time constraints without sacrificing the accuracy needed for safety- or quality-critical decisions. This is not a coincidence specific to these two problems — it reflects a structural trade-off that holds broadly across real-time computer-vision deployments.

7. Recommendations and Best Practices

- Always convert vague requirements (“fast enough”, “accurate enough”) into explicit numeric thresholds before comparing methods — this is what makes an objective, defensible decision possible.
- Treat each constraint as pass/fail, not as a weighted average — a method that is excellent on two criteria but fails a third is still not deployable in a hard-constraint system such as safety or quality control.
- Check the safety margin, not just pass/fail status — a method that barely clears a threshold (as YOLO does on accuracy in the factory case) should be monitored continuously in production, since real-world drift can push it back under the limit.
- Where the chosen method's margin is tight, consider periodic re-benchmarking, model retraining, or hardware upgrades as a contingency plan rather than assuming the initial benchmark numbers will hold indefinitely.
- For any future system with looser latency budgets (e.g., offline batch inspection or post-drive footage review), the heavyweight Deep Learning method should be reconsidered, since its accuracy advantage is only wasted — not disqualifying — once the latency constraint is removed.

8. Conclusion

Both the autonomous vehicle and smart factory case studies demonstrate that selecting a computer-vision method for a real-time, constraint-bound industrial system is an elimination process, not a simple “pick the best

score” exercise. In each case, two of the three candidate methods were disqualified — one for insufficient accuracy, the other for excessive latency and hardware demand — leaving YOLO as the only method that satisfied every constraint simultaneously. This consistent outcome across two independent industries illustrates why YOLO-class real-time detectors have become the de facto standard for latency-sensitive, resource-constrained computer-vision deployments: they occupy the practical middle ground between classical speed and deep-learning accuracy that hard, simultaneous constraints demand.

9. References

1. Viola, P., & Jones, M. (2001). Rapid Object Detection using a Boosted Cascade of Simple Features. IEEE CVPR.
2. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. IEEE CVPR.
3. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436–444.
4. Course material: CO4_AT2_Industry Problem Solving Task (provided problem statements and constraint data).